

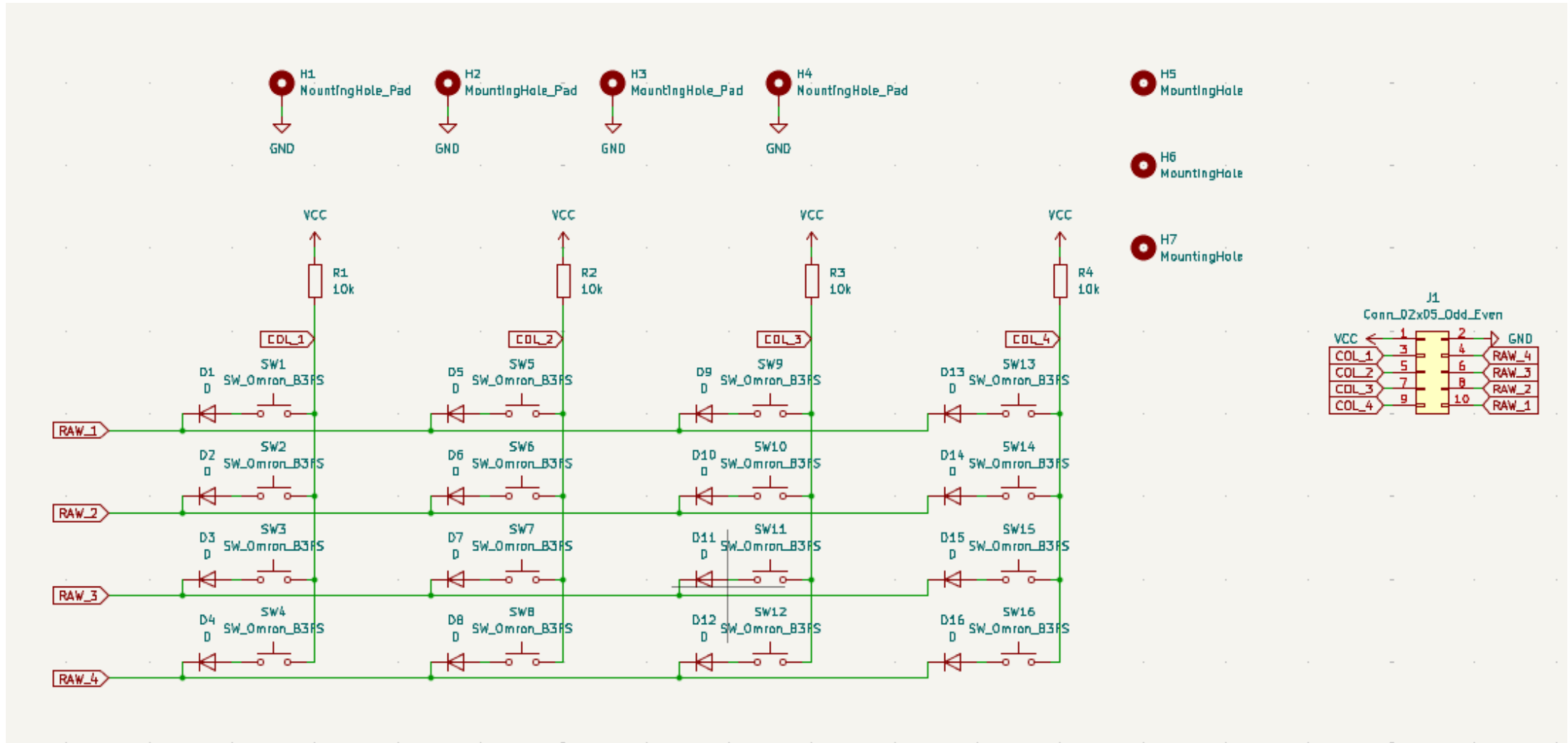
Labs

Stefan Cristi Zugravel
22/01/2026

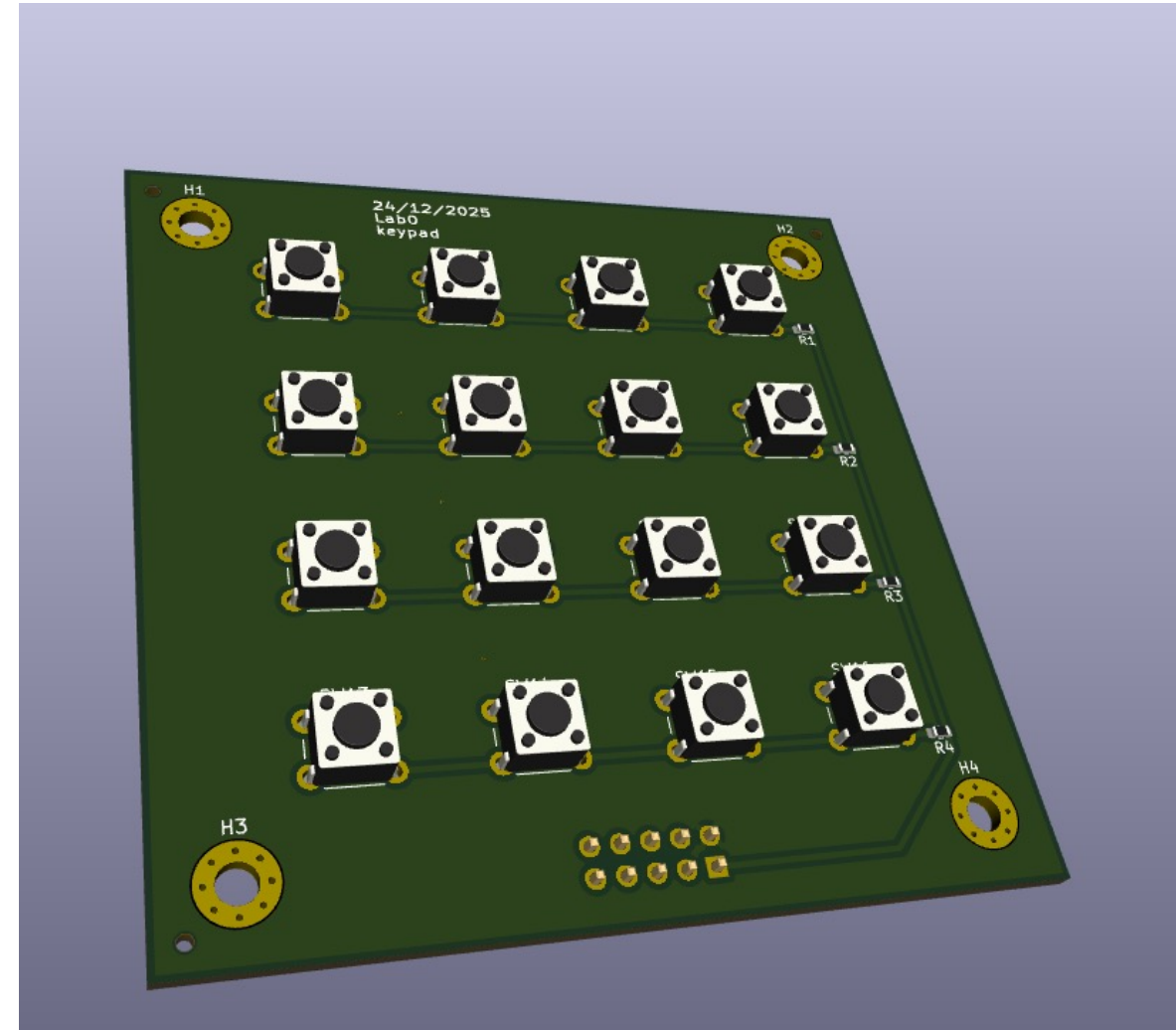
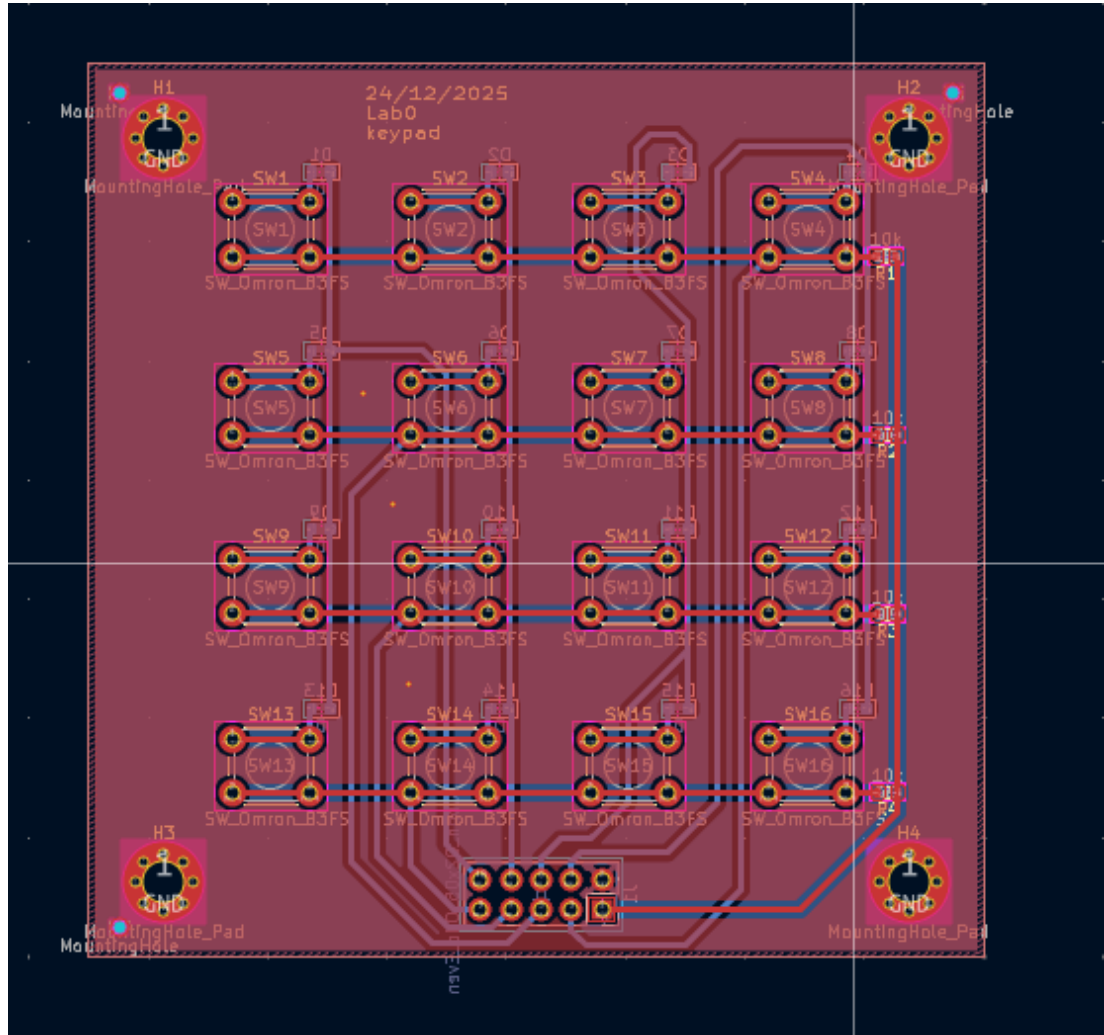
Lab 0

Keypad PCB with KiCAD

Schematic



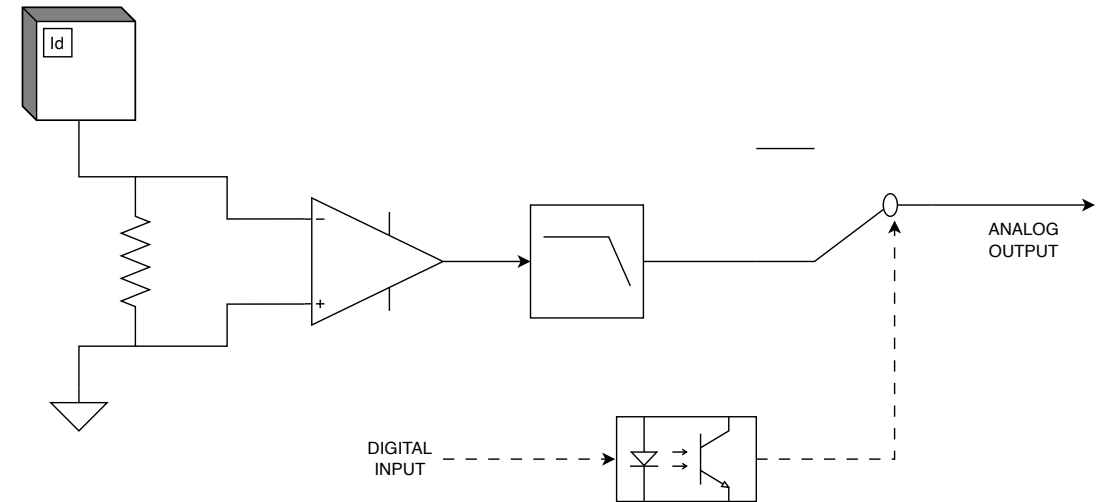
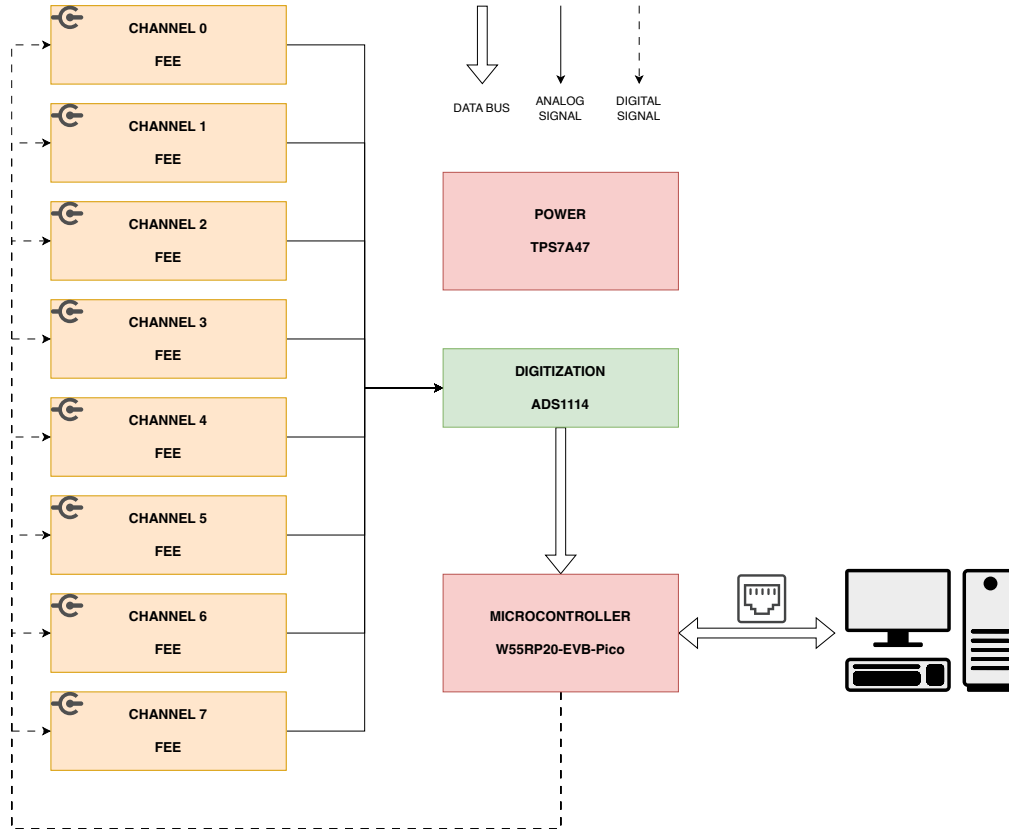
Layout



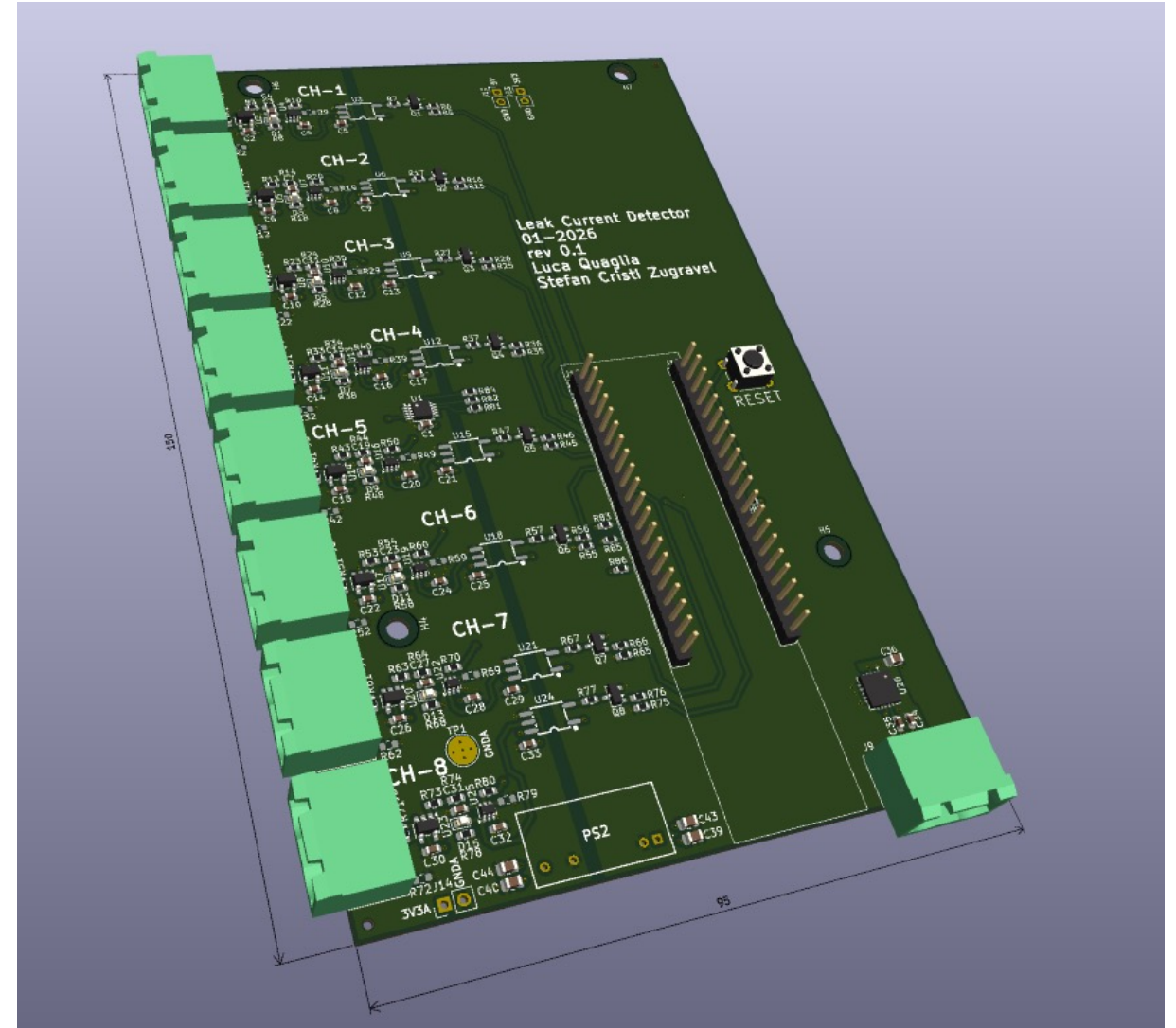
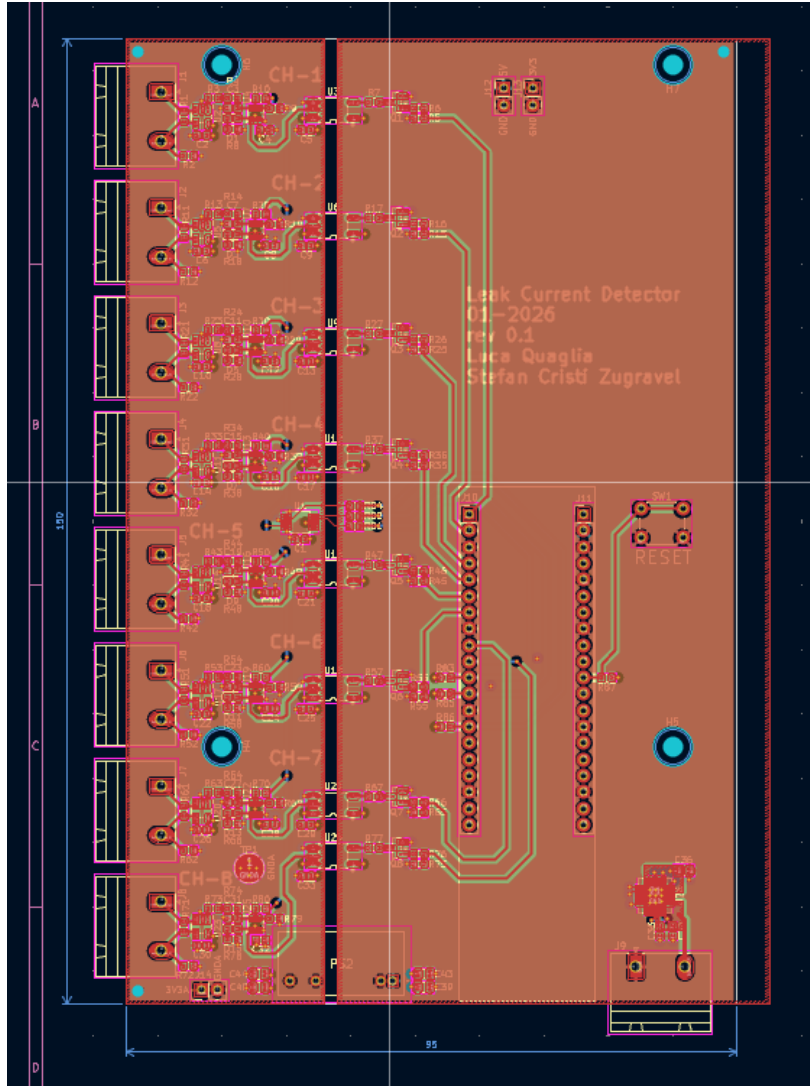
Lab 1

Current sensing PCB

Block Diagram



Layout and 3D model



Lab 2

Insert an FPGA in your PCB (quick preview)

Lab 3

Inverter, logic gates and mux in VHDL

1. open_vcd
2. log_vcd [get_object /<toplevel_testbench/uut/*>]
3. run *ns
4. close_vcd

```

[ ~/w/elettronica_base/1/lab3/lab3.si/s/b/xsim | main !28 ?4 python -m vcd2wavedrom.vcd2wavedrom
usage: vcd2wavedrom.py [-h] -i INPUT [-o OUTPUT] [-c CONFIGFILE] [-r SAMPLERATE] [-t MAXTIME] [-f OFFSET] [-z HSCALE] [--top]
vcd2wavedrom.py: error: the following arguments are required: -i/--input
[ ~/w/elettronica_base/1/lab3/lab3.si/s/b/xsim | main !28 ?4 python -m vcd2wavedrom.vcd2wavedrom -i dump.vcd -o dump.wavedrom
[ ~/w/elettronica_base/1/lab3/lab3.si/s/b/xsim | main !28 ?5 11
total 176
drwxr-xr-x 24 stefan staff 768B Jan 9 16:24 .
drwxr-xr-x 3 stefan staff 96B Dec 25 13:56 ..
-rw-r--r-- 1 stefan staff 832B Jan 7 09:41 compile.bat
-rwxr-xr-x@ 1 stefan staff 776B Jan 9 16:05 compile.sh
-rw-r--r--@ 1 stefan staff 286B Jan 9 16:08 dump.vcd
-rw-r--r-- 1 stefan staff 537B Jan 9 16:24 dump.wavedrom
-rw-r--r-- 1 stefan staff 957B Jan 7 09:41 elaborate.bat
-rw-r--r-- 1 stefan staff 468B Jan 9 16:05 elaborate.log
-rwxr-xr-x@ 1 stefan staff 844B Jan 9 16:05 elaborate.sh
-rw-r--r-- 1 stefan staff 891B Jan 7 09:41 simulate.bat
-rw-r--r-- 1 stefan staff 0B Jan 9 16:05 simulate.log
-rwxr-xr-x@ 1 stefan staff 778B Jan 9 16:05 simulate.sh
-rw-r--r-- 1 stefan staff 449B Dec 25 13:57 tb_inverter.tcl
-rw-r--r-- 1 stefan staff 3.9K Dec 25 14:05 tb_inverter_behav.wdb
-rw-r--r-- 1 stefan staff 449B Jan 7 09:41 tb_logic_gates.tcl
-rw-r--r-- 1 stefan staff 8.5K Jan 7 09:41 tb_logic_gates_behav.wdb
-rw-r--r-- 1 stefan staff 449B Jan 9 16:05 tb_mux.tcl
-rw-r--r-- 1 stefan staff 7.2K Jan 9 16:05 tb_mux_behav.wdb
-rw-r--r-- 1 stefan staff 238B Jan 9 16:05 tb_mux_vhdl.prj

```

Tool	Input Format(s)	Output Format(s)	Strengths	Limitations	Best Use Case
WaveDROM	JSON	SVG, PNG, HTML	Concise format, integrates well in web docs, portable, browser editor	Manual editing required unless auto-generated; limited analog support	Quick documentation, embedded diagrams
vcdvcd + Matplotlib	VCD	SVG, PNG, PDF	Full plotting control, integrates with Jupyter, supports analog signals	Requires Python skills; less compact format	Detailed diagrams from simulation/testbench
vcd2wavedrom	VCD	WaveDROM JSON	Bridges simulation and WaveDROM; CLI-based	Parsing is fragile on complex VCDs; limited maintenance	Auto-generating WaveDROM from simulation output
GTKWave	VCD, FST, LXT, etc.	Screen display, PDF	Fast, mature, interactive GUI; deep introspection of signals	Not designed for polished documentation output	Debugging HDL simulations
PulseView	Sigrok session files	Screen display	Broad hardware support, protocol decoders, user-friendly GUI	Limited export features; focused on real-world logic analyzer data	Capturing real signals and basic visualization
SchemDraw	Python script	SVG, PNG, PDF	Circuit + timing diagrams; LaTeX style drawing	Less intuitive than WaveDROM for timing; more verbose	Combining logic symbols and timing in documents
VCDRom	VCD	Browser display	No install needed; fast and simple viewer	View-only; minimal interactivity or customization	Quick online VCD viewing
TikZ-Timing	LaTeX script	PDF	Highly customizable, integrates with LaTeX	Steep learning curve; hard to preview	Academic papers, publication-quality diagrams

Lab 4

Length tuning and de-skewing in KiCAD

Lab 5

BCD counter in VHDL and different ways of dividing clock

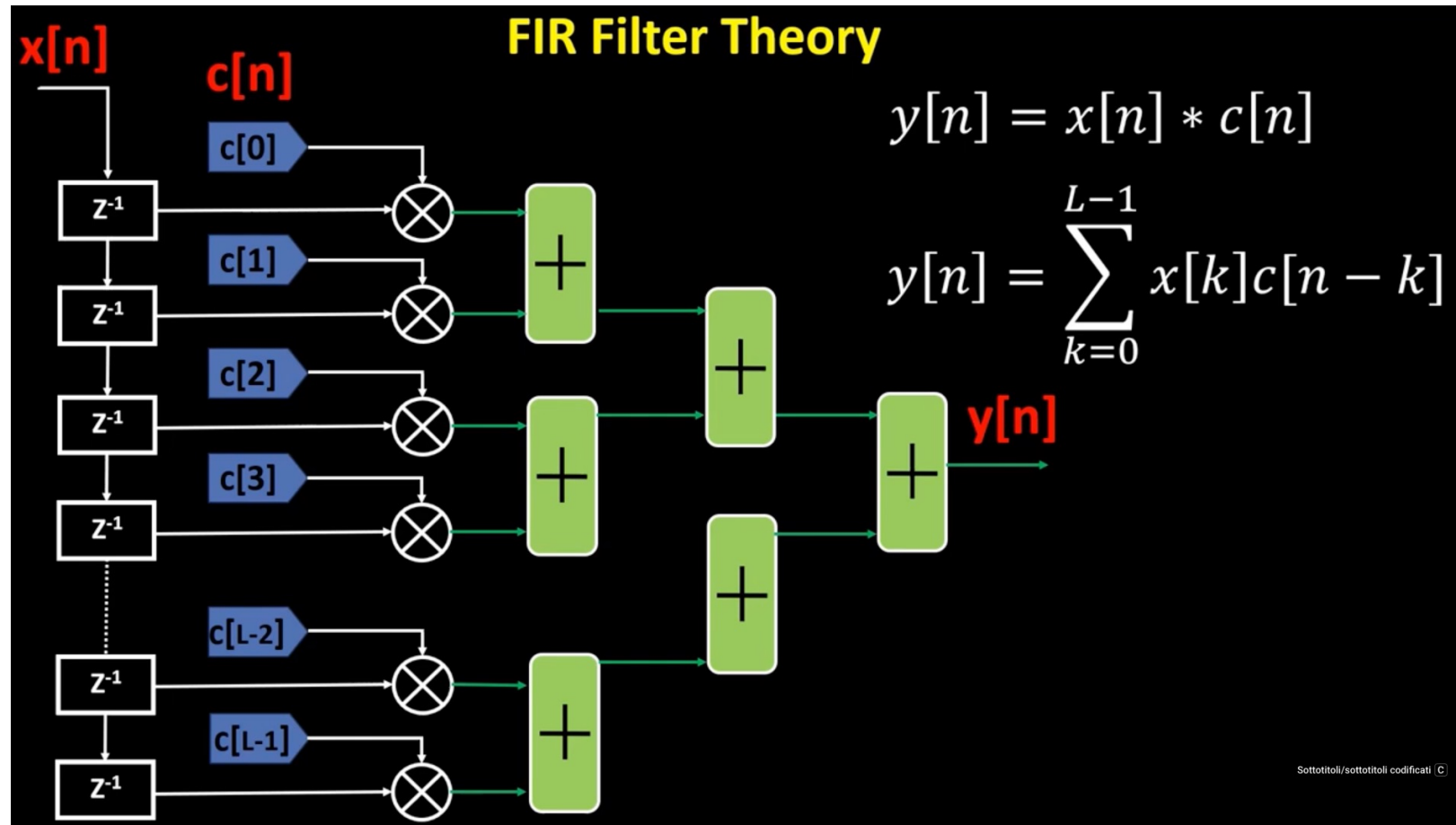
Lab 6

LTspice labs

Lab 7

FIR filter and DDS compiler using FPGA

The filter architecture

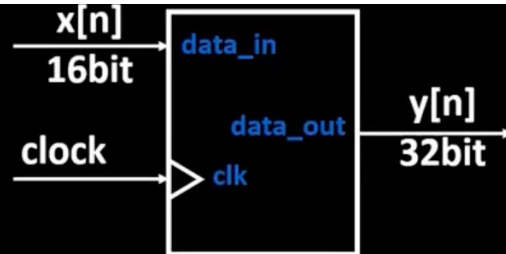



```

entity FIR_Filter is
  Port (
    clk      : in  std_logic;
    data_in   : in  std_logic_vector(15 downto 0);
    data_out  : out std_logic_vector(31 downto 0)
  );
end FIR_Filter;

architecture Behavioral of FIR_Filter is

```

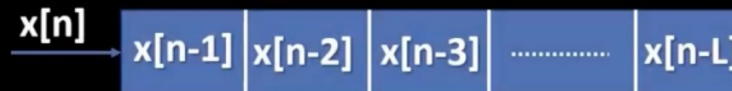


➤ Implement “delayed line”

```

-- =====
-- Delay Line: Shifts input samples through the buffer
-- Each new input moves the existing samples one stage down
-- =====
process(clk)
begin
  if rising_edge(clk) then
    for i in 31 downto 1 loop
      delay_line(i) <= delay_line(i - 1);
    end loop;
    delay_line(0) <= signed(data_in);
  end if;
end process;

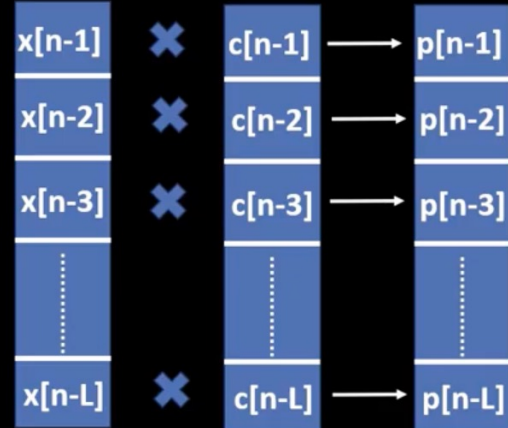
```



Provided VHDL code

➤ Next process implements the multiplier in parallel

```
-- =====  
-- Multiply Stage: Multiplies each delayed sample with its coefficient  
-- Produces 32 parallel product values for summing  
-- =====  
process(clk)  
begin  
    if rising_edge(clk) then  
        for i in 0 to 31 loop  
            products(i) <= delay_line(i) * coeffs(i);  
        end loop;  
    end if;  
end process;
```



Lab 8

FPGA PWM controller used for LED breathing effect

